In [2]:
```
# Import required libraries
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# 1. Load the Breast Cancer Wisconsin dataset
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Dataset shape:", X.shape)
print("\nFirst five rows:")
print(X.head())

print("\nTarget classes:")
print(data.target_names)
print("0 = malignant, 1 = benign")

# 2. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

# 3. MLE: Logistic Regression without regularization
mle_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        penalty=None,
        solver="lbfgs",
        max_iter=10000
    ))
])

mle_model.fit(X_train, y_train)
mle_predictions = mle_model.predict(X_test)

# 4. MAP with L1 regularization
l1_pipeline = Pipeline([
    ("scaler", StandardScaler()),
```

```
    ("model", LogisticRegression(
        penalty="l1",
        solver="saga",
        max_iter=20000,
        random_state=42
    ))
])

# Tune inverse regularization strength using 5-fold cross-validation
param_grid = {
    "model__C": np.logspace(-3, 2, 10)
}

l1_search = GridSearchCV(
    estimator=l1_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

l1_search.fit(X_train, y_train)

l1_model = l1_search.best_estimator_
l1_predictions = l1_model.predict(X_test)

# 5. MAP with L2 regularization
l2_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        penalty="l2",
        solver="lbfgs",
        max_iter=10000
    ))
])

l2_search = GridSearchCV(
    estimator=l2_pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

l2_search.fit(X_train, y_train)

l2_model = l2_search.best_estimator_
l2_predictions = l2_model.predict(X_test)

# 6. Compare model performance
models = {
    "MLE (No Regularization)": (mle_model, mle_predictions),
    "MAP with L1": (l1_model, l1_predictions),
    "MAP with L2": (l2_model, l2_predictions)
}

results = []

for name, (model, predictions) in models.items():
    results.append({
```

```
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1-Score": f1_score(
            y_test, predictions, zero_division=0
        )
    })

results_df = pd.DataFrame(results)

print("\nMODEL PERFORMANCE COMPARISON")
print(results_df.round(4).to_string(index=False))

# 7. Display the best hyperparameters
print("\nBEST HYPERPARAMETERS")
print("Best L1 C:", l1_search.best_params_["model__C"])
print("Best L2 C:", l2_search.best_params_["model__C"])

print("\nBest L1 cross-validation accuracy:",
      round(l1_search.best_score_, 4))

print("Best L2 cross-validation accuracy:",
      round(l2_search.best_score_, 4))

# 8. Compare estimated coefficients
mle_coef = mle_model.named_steps["model"].coef_[0]
l1_coef = l1_model.named_steps["model"].coef_[0]
l2_coef = l2_model.named_steps["model"].coef_[0]

coef_comparison = pd.DataFrame({
    "Feature": data.feature_names,
    "MLE Coefficient": mle_coef,
    "L1 Coefficient": l1_coef,
    "L2 Coefficient": l2_coef
})

print("\nFIRST 10 PARAMETER ESTIMATES")
print(coef_comparison.head(10).round(4).to_string(index=False))

# 9. Analyze coefficient magnitudes and sparsity
print("\nPARAMETER ANALYSIS")

for name, (model, _) in models.items():
    coefficients = model.named_steps["model"].coef_[0]

    print(f"\n{name}")
    print("Sum of absolute coefficients:",
          round(np.sum(np.abs(coefficients)), 4))
    print("L2 coefficient norm:",
          round(np.linalg.norm(coefficients), 4))
    print("Number of non-zero coefficients:",
          np.count_nonzero(np.abs(coefficients) > 1e-8))

# 10. Classification reports
for name, (_, predictions) in models.items():
```

```
    print(f"\nCLASSIFICATION REPORT: {name}")
    print(classification_report(
        y_test,
        predictions,
        target_names=data.target_names,
        zero_division=0
    ))

print("\nExperiment completed successfully.")
```

```
Dataset shape: (569, 30)

First five rows:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness
\
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst perimet
er  \
0                 0.07871  ...         25.38          17.33           184.
60
1                 0.05667  ...         24.99          23.41           158.
80
2                 0.05999  ...         23.57          25.53           152.
50
3                 0.09744  ...         14.91          26.50            98.
87
4                 0.05883  ...         22.54          16.67           152.
20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   worst concave points  worst symmetry  worst fractal dimension
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 30 columns]

Target classes:
['malignant' 'benign']
0 = malignant, 1 = benign

Training samples: 455
Testing samples: 114

MODEL PERFORMANCE COMPARISON
                  Model  Accuracy  Precision  Recall  F1-Score
MLE (No Regularization)    0.9211     0.9701  0.9028    0.9353
           MAP with L1    0.9825     0.9861  0.9861    0.9861
           MAP with L2    0.9825     0.9861  0.9861    0.9861
```

```
BEST HYPERPARAMETERS
Best L1 C: 0.5994842503189409
Best L2 C: 0.5994842503189409

Best L1 cross-validation accuracy: 0.9736
Best L2 cross-validation accuracy: 0.9802

FIRST 10 PARAMETER ESTIMATES
                Feature  MLE Coefficient  L1 Coefficient  L2 Coefficient
            mean radius          17.5773          0.0000         -0.5037
           mean texture         -98.5093         -0.4195         -0.5700
         mean perimeter          15.6478          0.0000         -0.4748
              mean area         -35.5952          0.0000         -0.5133
        mean smoothness         -53.5956          0.0000         -0.2197
       mean compactness         360.4122          0.0000          0.4059
         mean concavity         -98.4887          0.0000         -0.5192
    mean concave points         -35.3375         -0.3759         -0.6227
          mean symmetry         -19.3169          0.0000         -0.1743
mean fractal dimension        -107.3546          0.0000          0.2474

PARAMETER ANALYSIS

MLE (No Regularization)
Sum of absolute coefficients: 2729.3958
L2 coefficient norm: 681.0459
Number of non-zero coefficients: 30

MAP with L1
Sum of absolute coefficients: 11.0312
L2 coefficient norm: 3.9998
Number of non-zero coefficients: 13

MAP with L2
Sum of absolute coefficients: 14.9884
L2 coefficient norm: 3.0951
Number of non-zero coefficients: 30

CLASSIFICATION REPORT: MLE (No Regularization)
              precision    recall  f1-score   support

   malignant       0.85      0.95      0.90        42
      benign       0.97      0.90      0.94        72

    accuracy                           0.92       114
   macro avg       0.91      0.93      0.92       114
weighted avg       0.93      0.92      0.92       114


CLASSIFICATION REPORT: MAP with L1
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114


CLASSIFICATION REPORT: MAP with L2
```

```
                precision    recall  f1-score   support

     malignant       0.98      0.98      0.98        42
        benign       0.99      0.99      0.99        72

      accuracy                           0.98       114
     macro avg       0.98      0.98      0.98       114
  weighted avg       0.98      0.98      0.98       114


Experiment completed successfully.
```

In [ ]: